

Structures de données et algorithmes

Projet 2: arbres binaires de recherches

3 avril 2026

Ce projet porte sur la partie structure de données du cours, en particulier les arbres binaires de recherche. On vous demande à la fois d'utiliser l'implémentation d'arbre vue en cours mais également de l'étendre pour résoudre un problème concret. Le projet peut être réalisé seul ou par **groupe de deux étudiants** maximum. Tous les fichiers associés au projet sont disponibles sur Ecampus.

1 Objectif

Une société de taxi de Porto au Portugal a collecté l'ensemble des trajets effectués par ses taxis pendant une année (voir la figure 1) et souhaiterait pouvoir stocker cette information pour déterminer pour une position donnée dans la ville la liste des trajets ayant démarré près de cette position (afin par exemple de déterminer l'endroit idéal où placer une nouvelle station de taxis ou de pouvoir indiquer à un client via une application les endroits où il a le plus de chance de rencontrer d'autres gens afin de partager un taxi). On vous demande de mettre au point ce système. Vous voulez faire de l'excès de zèle et souhaiter minimiser au maximum le temps de réponse du système à une requête.

De manière plus abstraite, on souhaite mettre au point une structure de données de type dictionnaire permettant de stocker un ensemble potentiellement important de paires clé-valeur où la clé est une position (x, y) dans le plan et la valeur associée est quelconque. Ce système doit être capable de répondre à deux types de requêtes :

- Renvoyer la valeur associée à une paire donnée (x_q, y_q) si elle a été stockée dans la structure (*recherche exacte*)
- Renvoyer l'ensemble des valeurs associées à des clés situées à une distance (euclidienne) maximale d_{max} d'une position (x_q, y_q) données (*recherche dans une boule*).

On supposera dans ce projet que la structure est statique, c'est-à-dire que la liste de toutes les positions à stocker dans la structure est connue initialement et n'est pas amenée à évoluer. En conséquence, la structure ne devra pas implémenter d'opérations d'insertion ou de suppression de positions.

On se propose d'étudier trois implémentations de cette structure.

Par liste. La première approche consiste à stocker les positions et les valeurs associées dans une liste (ou plusieurs) et à effectuer une recherche simple linéaire dans cette liste pour répondre aux deux types de requêtes.

Par arbre binaire de recherche simple. La deuxième approche consiste à stocker les données dans un arbre binaire de recherche standard en utilisant comme valeur de clé, un encodage de Morton¹ (ou courbe en Z) des coordonnées. Cet encodage, noté $Z(x, y)$, garantit que si $x_{min} \leq x \leq x_{max}$ et $y_{min} \leq y \leq y_{max}$, alors $Z(x_{min}, y_{min}) \leq Z(x, y) \leq Z(x_{max}, y_{max})$. L'implémentation de la recherche dans une boule se réduit alors à déterminer les coordonnées (x_{min}, y_{min}) et (x_{max}, y_{max}) respectivement des coins inférieure gauche et supérieur droit d'un carré entourant cette boule et à rechercher parmi les positions de l'arbre entre $Z(x_{min}, y_{min})$ et $Z(x_{max}, y_{max})$ celles qui se trouvent effectivement dans la boule.

1. Voir par exemple https://en.wikipedia.org/wiki/Z-order_curve

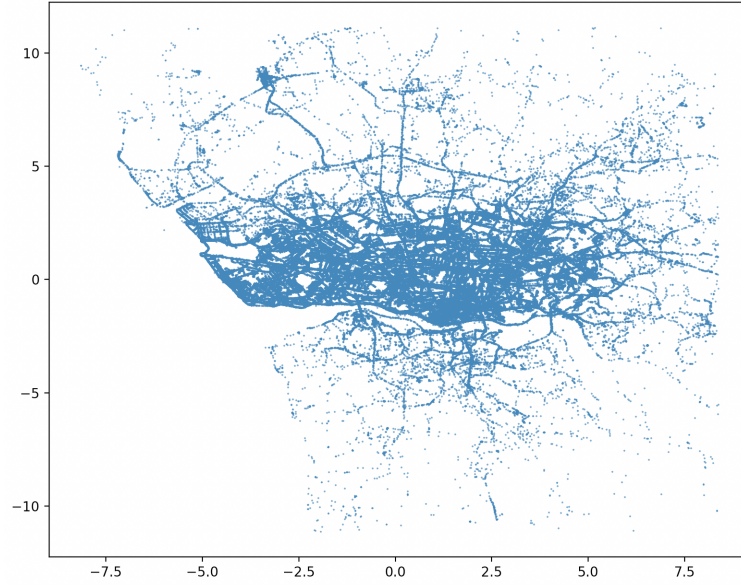


FIGURE 1 – Chaque point représente la position de départ d’une course d’un des 442 taxis de la ville de Porto effectuée entre le 1/07/2013 et le 30/06/2014. Les données ont été obtenues de Kaggle

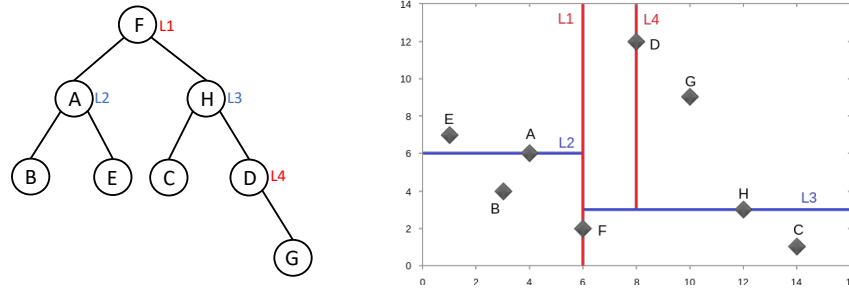


FIGURE 2 – Un arbre binaire de recherche à deux dimensions et le découpage correspondant de l’espace des coordonnées. (source : Wikipedia)

Par arbre binaire de recherche à deux dimensions. La seconde approche, consiste à stocker les villes dans un arbre binaire de recherche adapté pour prendre en compte directement des clés (x, y) à deux dimensions². Cette structure est très proche de la structure d’arbre binaire de recherche vue au cours si ce n’est que la fonction de comparaison change à chaque profondeur dans l’arbre. A la profondeur 0 (la racine), les clés (x, y) sont comparées sur la coordonnée x , à la profondeur 1 sur la coordonnée y , à la profondeur 2 sur la coordonnée x à nouveau et ainsi de suite. En dehors de cette modification, les opérations de recherche et d’insertion sont similaires à celles implémentées pour les arbres binaires de recherche classiques (voir la figure 2 pour une illustration). L’avantage de cette structure est qu’elle rend plus efficace l’implémentation de la fonction de recherche dans une boule.

Optimisation des structures d’arbres. Étant donné qu’on dispose d’emblée de toutes les positions à stocker dans le dictionnaire, plutôt que d’insérer les clés une à une dans un ordre arbitraire dans un arbre initialement vide, il est possible de s’arranger pour construire directement l’arbre, que ce soit l’arbre binaire standard ou l’arbre binaire à deux dimensions, de façon à en minimiser autant que possible la profondeur. Dans les deux cas, cela consiste à positionner en chaque nœud une clé permettant de placer autant de positions à sa gauche qu’à sa droite. L’arbre de la figure 2 est un arbre de hauteur minimal

2. Cette idée peut se généraliser facilement à $d > 2$ dimensions.

étant donné le nombre de positions.

2 Implémentation

Pour implémenter ces idées, nous vous fournissons les codes suivants :

- `Point.h/.c` : un type de données correspondant à une position (x, y) permettant de calculer la distance entre deux points et de les comparer.
- `List.h/.c` : un type de données correspondant à une liste générique implémentée par une liste liée simple.
- `BST.h/.c` : une implémentation (partielle) générique d'arbre binaire de recherche
- `PointDct.h` : l'interface de la structure de dictionnaire utilisée pour stocker les positions et implémentant les deux types de requête (recherches exactes et dans un boule).
- `PointDctList.c` : l'implémentation par liste de cette structure.

Votre tâche est d'ajouter certains fonctions dans le fichier `BST.c`, d'implémenter la solution par arbre binaire de recherche dans `PointDctBST.c` et par arbre à deux dimensions dans le fichier `PointDctBST2d.c`. Quelques précisions sont données ci-dessous sur les implémentations attendues. Consultez bien les commentaires dans les fichiers `.h` pour connaître les contraintes d'implémentation de chaque fonction.

Sauf mention contraire ci-dessous, toutes les fonctions doivent être implémentées de la manière la plus efficace possible.

Arbre binaire de recherche (BST.c). Vous devez ajouter dans le fichier `BST.c` trois fonctions (voir `BST.h` pour une description détaillée de ces fonctions) :

- `bstOptimalBuild` : une fonction permettant de construire un arbre binaire de recherche de hauteur minimale à partir d'une liste de clés et d'une liste de valeurs.
- `bstRangeSearch` : une fonction renvoyant une liste liée contenant les valeurs associées aux clés k de l'arbre telles que $k_{min} \leq k \leq k_{max}$, où k_{min} et k_{max} sont deux arguments de la fonction.
- `bstAverageNodeDepth` : une fonction calculant la profondeur moyenne des clés dans l'arbre (telle que définie au cours).

Ces fonctions vous serviront pour implémenter les fonctions de l'interface `PointDct.h`.

Dictionnaires de points. Les fichiers `PointDctBST.c` et `PointDctBST2d.c` doivent implémenter les fonctions de l'interface `PointDct.h` en utilisant respectivement un arbre binaire de recherche et un arbre binaire de recherche à deux dimensions, tel que décrit plus haut. Les fonctions à implémenter incluent :

- `pdctCreate` : stockant dans la structure les positions et valeurs données en argument. Dans le cas des deux types d'arbres, vous devez créer une arbre de hauteur minimale, afin de rendre les opérations de recherche les plus efficaces possible.
- `pdctExactSearch` : implémentant la recherche exacte.
- `pdctBallSearch` : implémentant la recherche dans une boule.
- `pdctAverageNodeDepth` et `pdctHeight` : renvoyant respectivement la profondeur moyenne d'un nœud de l'arbre et la hauteur de l'arbre. Cette fonction n'est pas nécessaire à l'implémentation. Elle ne vous est demandée que dans le but de pouvoir tester votre code.

Fichier `PointDctBST.c`. Vous devrez utiliser l'implémentation d'arbre binaire générique des fichiers `BST.c/.h` pour implémenter ces fonctions. Une fonction implémentant la code de Morton $Z(x, y)$ vous est fournie dans le fichier `PointDctBST.c`. Cette fonction prend comme argument des entiers non signés de 32 bits et renvoie la valeur Z comme un entier non signé de 64 bits. Il est donc nécessaire de ramener les coordonnées x et y de chaque point à une valeur entre 0 et `UINT32_MAX` (le plus grand entier de 32 bits). Étant donné cette discrétisation, il est possible que deux coordonnées différentes puissent correspondre au même Z . Vous devrez prendre ça en compte pour l'implémentation de la recherche exacte.

Fichier `PointDctBST2d.c`. La structure d'arbre binaire de recherche à deux dimensions devra être définie dans ce fichier directement. Il existe plusieurs manière de construire l'arbre de profondeur minimale à partir de la liste de positions. Essayez d'obtenir l'implémentation la plus efficace possible. Pour

implémenter la recherche dans une boule, l'idée est de parcourir l'arbre en profondeur d'abord, mais en arrêtant l'exploration d'une branche dès qu'on peut déterminer qu'elle ne peut contenir aucune position située dans la boule. Pour déterminer ça, on exploitera le fait que les points à gauche (respectivement à droite) d'un nœud ont tous des valeurs de la coordonnée comparée au nœud (x ou y selon la profondeur du nœud) strictement plus petites (resp. plus grandes ou égales) que la position stockée en ce nœud.

Fichiers de test (testcputime.c et testtaxi.c). Le fichier `testcputime.c` fournit une fonction `main` permettant de tester chaque implémentation sur un ensemble de positions générées aléatoirement uniformément dans $[0, 0; 1, 0] \times [0, 0; 1, 0]$. Les arguments en ligne de commande de l'exécutable généré sont le nombre de points à stocker dans le dictionnaire, le nombre de recherche à effectuer et le rayon de ces recherches.

Le fichier `testtaxi.c` fournit une fonction `main` permettant de tester vos implémentations sur la base de données des positions de départ des trajets de taxis de la ville de Porto (fichier `taxitripsporto.csv`). L'exécutable prend en argument les longitudes et latitudes du centre de la boule³ et le rayon (en km) pour la recherche. Notez que les positions dans la structure ne sont pas représentées directement par les longitudes et latitudes parce que les distances euclidiennes entre des paires de longitudes et latitudes ne représentent pas directement les distances dans le système métrique entre ces points. Une transformation est nécessaires pour donner des coordonnées (x, y) qui préservent ces distances. Pour traiter ces longitudes et latitudes, le fichier `Point.h` fournit une fonction `ptNewFromLL` qui crée une position à partir des longitudes et latitudes. Inversement, vous pouvez obtenir les longitudes et latitudes d'un point avec les fonctions `ptGetLongitude` et `ptGetLatitude`. Ces fonctions vous seront utiles pour répondre à la dernière question dans le rapport.

Un `Makefile` vous est fourni pour automatiser la compilation. Il génère par défaut (commande `make` ou `make all`) 3 exécutables, `testlist`, `testbst` et `testbst2d` pour chaque implémentation de dictionnaire. Pour générer un exécutable à partir de `testtaxi.c`, vous devez utiliser la commande `make testtaxi`. Par défaut, l'implémentation par liste est utilisée pour ce dernier. Vous devrez modifier le `makefile` pour utiliser la structure la plus appropriée.

3 Analyse théorique et empirique

Dans le rapport, nous vous demandons de répondre aux questions suivantes :

1. Donnez le pseudo-code de la fonction `bstOptimalBuild`. Analysez sa complexité en temps dans le pire et le meilleur cas en fonction du nombre de clés N .
2. Expliquez comment vous avez implémenté `pdCreate` dans le cas de l'arbre binaire de recherche 2d. Analysez sa complexité en temps dans le pire et le meilleur cas en fonction du nombre de positions N .
3. Donnez le pseudo-code de la fonction `bstRangeSearch`. Analysez sa complexité en temps dans le pire et le meilleur cas en fonction du nombre total de clés N et du nombre de clés dans l'intervalle k , en supposant que l'arbre a été construit par la fonction `bstOptimalBuild`.
4. Comparez empiriquement les temps de calcul des trois approches, à la fois pour la construction du dictionnaire et la réponse à des requêtes de type exactes et de type boules. Étudiez l'impact du nombre de points stockés dans la structure et du rayon de la recherche. Commentez brièvement vos résultats.
Suggestion : Faites deux expériences. Fixez le nombre de recherches à 10^4 et le rayon à 0.01 et faites varier le nombre de points dans le dictionnaire dans $\{10^4, 10^5, 10^6\}$. Fixez ensuite le nombre de requêtes à 10^4 et le nombre de points dans le dictionnaire à 10^5 et faites varier le rayon dans $\{0.01, 0.05, 0.1\}$
5. Concluez quant à l'intérêt relatif des trois approches.
6. Utilisez l'implémentation la plus efficace pour trouver l'endroit à Porto qui a la densité de taxis la plus élevée. Définissez pour cela une grille de points, entre les coordonnées x et y (après transformation) minimales et maximales, espacés de 100m selon chaque coordonnée et calculez le

3. Le centre de la ville de Porto est situé à une longitude et latitude respectives de -8.6291 et 41.1579.

nombre de taxis partant de chaque point de la grille dans un rayon de 100m également. Précisez le nombre de taxis maximal et la position (en terme de longitude et latitude) trouvée. Utilisez par exemple *Google Maps* pour déterminer à quel endroit à Porto correspond cette position.

4 Deadline et soumission

Le projet est à réaliser par **groupe de deux étudiants maximum** pour le **3 mai 2026, à 23h59** au plus tard. Il doit être soumis via Gradescope (<http://gradescope.com>, code cours **YBZ7RR**).

Les fichiers suivants doivent être remis :

1. Votre rapport au format PDF. Soyez bref mais précis et respectez bien la numération des questions.
2. Les fichiers suivants (uniquement) : `BST.c`, `PointDctBST.c`, et `PointDctBST2d.c`.
3. Le fichier `declaration.txt` rempli, où vous indiquerez les contributions des différents membres du groupe et certifierez avoir respecté les règles du cours en matière de plagiat et d'utilisation d'outils d'IA consultables sur Ecampus. Ce fichier doit être remis avec le code sur Gradescope..

Respectez bien les extensions de fichiers ainsi que les noms des fichier `*.c` (en ce compris les minuscules/majuscules). N'incluez aucun fichier supplémentaire. Ne soumettez qu'un seul projet par groupe en associant tous les membres du groupe à la soumission.

Vos fichiers seront évalués avec les flags de compilation habituels (`--std=c99 --pedantic -Wall -Wextra -Wmissing-prototypes`) qui sont aussi utilisés dans le fichier `Makefile`. La présence de *warnings* impactera négativement la cote finale. Un fichier qui ne compile pas recevra une cote nulle.

Un projet non rendu à temps recevra une cote globale nulle. La soumission du projet suppose que vous avez pris connaissance des règles en terme de plagiat rappelées sur Ecampus. En cas de plagiat ⁴ avéré, le groupe se verra affecter une cote nulle à l'ensemble du cours.

Les critères de correction sont précisés sur Ecampus.

Bon travail !

4. Des tests anti-plagiat seront réalisés.